МИНОБРНАУКИ РОССИИ

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ

ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)

Кафедра Робототехники и автоматизации производственных систем

ОТЧЕТ

по лабораторной работе № 4

по дисциплине «Техническое зрение и специальное программное обеспечение»

ТЕМА: ПОИСК ГЕОМЕТРИЧЕСКИХ ФИГУР НА ИЗОБРАЖЕНИЯХ.

Студент гр. 1406 _____ Бондаренко Ю.В.

Преподаватель _____ Федоркова А.О.

Санкт-Петербург

2026

Цель работы - изучить методы поиска прямых и окружностей с помощью преобразования Хафа и нахождение контуров на изображении.

Задание №1

Для «удаления» линий их сперва необходимо найти с помощью преобразования Хафа, после чего по их определённым параметрам нанести на их месте белые линии (рисунки 1 и 2). Код для выполнения этих действий:

```
image = cv2.imread('line5.jpg')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
gray = cv2.GaussianBlur(gray, (5, 5), 0)
edges = cv2.Canny(gray, 50, 150, apertureSize=3)
lines = cv2.HoughLines(edges, 1, np.pi / 180, 200)
for line in lines:
    rho, theta = line[0]
    a = np.cos(theta)
    b = np.sin(theta)
    x0 = a * rho
    y0 = b * rho
    x1 = int(x0 + 1000 * (-b))
    y1 = int(y0 + 1000 * (a))
    x2 = int(x0 - 1000 * (-b))
    y2 = int(y0 - 1000 * (a))
    cv2.line(image, (x1, y1), (x2, y2), (255, 255, 255), 14)
cv2.imshow('Hough Lines', image)
cv2.imshow('ISH', gray)
cv2.imshow('edges', edges)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

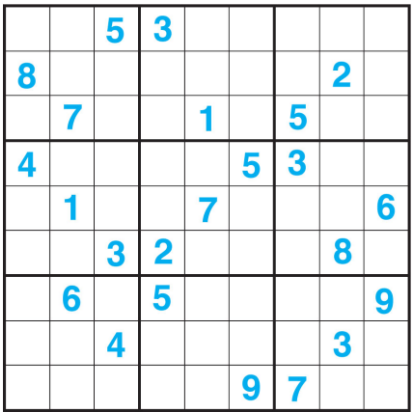

Рисунок 1 – Исходное изображение с линиями

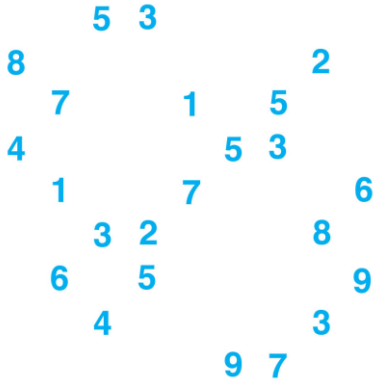

Рисунок 2 – Изображение с убранными линиями

Задание №2

Окружности на изображении (рисунок 3) обнаруживаются соответствующим преобразованием Хафа (рисунок 4), откуда становятся известны радиусы, среди которых ищется максимум (рисунок 5)

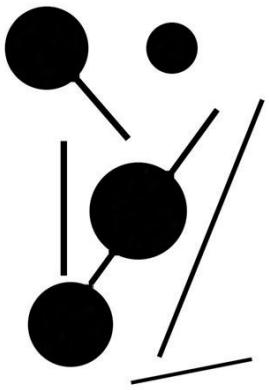

Рисунок 3 – Исходное изображение

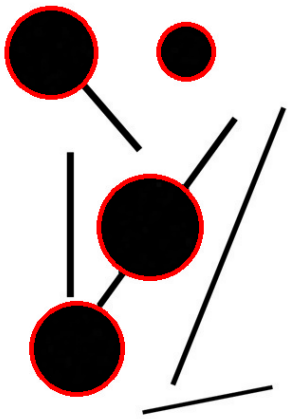

Рисунок 4 – Изображение с выделенными окружностями

```
[[106.5 412.5  48.3]
 [183.5 284.5  54.6]
 [221.5  99.5  29.3]
 [ 79.5 100.5  47.4]]
Максимальный радиус равен 54
```

Рисунок 5 – Вывод максимального радиуса

Код для получения указанных результатов:

```
path = r"./res_circ.jpg"
pic = cv2.imread(path, cv2.IMREAD_COLOR)
#b, r, g = cv2.split(pic)
#pic = cv2.merge([b,r,g])
pic_g = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
pic_f = cv2.GaussianBlur(255 - pic_g,(3,3),0)
a = pic.shape[0]
circles = cv2.HoughCircles(pic_g, cv2.HOUGH_GRADIENT, 1, minDist =
a//20, param1 = 100, param2 = 50, minRadius = a//1000, maxRadius =
a//2)
print(circles[0])
max_radius = 0
for circle in circles[0]:
    center = int(circle[0]), int(circle[1])
    radius = int(circle[2])
    if max_radius < radius: max_radius = radius
    cv2.circle(pic, center, radius, (0, 0, 255), 3)
cv2.imshow('result', pic)
cv2.imshow('original', pic_g)
```

```
cv2.imshow('pic_f', pic_f)
print(f"Максимальный радиус равен {max_radius}")
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Поиск максимальной длины отрезка выполняется аналогично с помощью требуемого преобразования Хафа – рисунки 6 и 7.

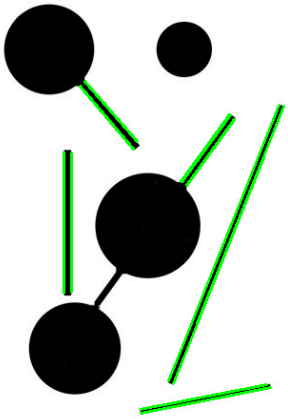

Рисунок 6 – Изображение с выделенными отрезками

```
Максимальная длина отрезка: 311.4113035841827
```

Рисунок 7 – Вывод максимальной длины

Код для получения указанных результатов:

```
path = r"./res_circ.jpg"
image = cv2.imread(path)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
edges = cv2.Canny(blurred, 50, 150, apertureSize=3)
lines = cv2.HoughLinesP(edges, 1, np.pi / 180, threshold=50,
minLineLength=30, maxLineGap=5)
max_len = 0
for num, line in enumerate(lines):
    x1, y1, x2, y2 = line[0]
    len = np.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)
    if max_len < len: max_len = len
    cv2.line(image, (x1, y1), (x2, y2), (0, 255, 0), 2)
cv2.imshow('Probabilistic Hough Lines', image)
cv2.imshow('edges', edges)
```

```
cv2.imshow('gray', gray)
print(f"Максимальная длина отрезка: {max_len}")
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Задание №3

Для выделения знаков (рисунки 8-10) используется следующий код, в котором меняется лишь путь к файлу и параметры макс. и мин. радиусов:

```
path = r"./4_5_1.jpg"
pic = cv2.imread(path, cv2.IMREAD_REDUCED_COLOR_8)
#b, r, g = cv2.split(pic)
#pic = cv2.merge([b,r,g])
pic_g = cv2.imread(path, cv2.IMREAD_REDUCED_GRAYSCALE_8)
pic_f = cv2.GaussianBlur(255 - pic_g,(3,3),0)
a = pic.shape[0]
circles = cv2.HoughCircles(pic_g, cv2.HOUGH_GRADIENT, 1, minDist =
a//20, param1 = 100, param2 = 50, minRadius = a//1000, maxRadius =
a//15)
print(circles[0])
for circle in circles[0]:
    center = int(circle[0]), int(circle[1])
    radius = int(circle[2])
    cv2.circle(pic, center, radius, (0, 0, 255), 3)
cv2.imshow('result', pic)
cv2.imshow('original', pic_g)
cv2.imshow('pic_f', pic_f)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Рисунок 8 – Обнаруженный знак №1

Рисунок 9 – Обнаруженный знак №2

Рисунок 10 – Обнаруженный знак №3

Задание №4

Для определения типа контура можно использовать сравнение его площади с описываемыми вокруг него окружностью или прямоугольником:

```
path = r"./6-1.png"
def contourRecogniton(contour, maxDiff):
    contourArea = cv2.contourArea(contour=contour)
    c, r = cv2.minEnclosingCircle(contour)
    circArea = np.pi * r**2
    rect = cv2.boundingRect(contour)
    x,y,w,h = rect
    rectArea = w * h
    if abs(contourArea - circArea) <= maxDiff:
        return "Окружность"
    if abs(contourArea - rectArea) <= maxDiff:
        return "Прямоугольник"
    else:
        return "Треугольник"
picture =cv2.imread(path,cv2.IMREAD_COLOR)
gray = cv2.cvtColor(picture,cv2.COLOR_BGR2GRAY)
gray = cv2.bitwise_not(gray)
res = cv2.findContours(gray,cv2.RETR_TREE,cv2.CHAIN_APPROX_SIMPLE)
contours, hierarchy = res
cv2.drawContours(image=picture, contours=contours,
contourIdx=2,color=100,thickness=5)
for index,contour in enumerate(contours):
    contourType = contourRecogniton(contour,1500)
    match contourType:
        case "Окружность":
            cv2.drawContours(image=picture, contours=contours,
contourIdx=index,color=(255,0,0),thickness=5)
        case "Прямоугольник":
             cv2.drawContours(image=picture, contours=contours,
contourIdx=index,color=(0,0,255),thickness=5)
        case "Треугольник":
```

```
            cv2.drawContours(image=picture, contours=contours,
contourIdx=index,color=(255,0,127),thickness=5)
cv2.imshow("picture", picture)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

В результате изображение на рисунке 11 преобразуется к виду на рисунке 12.

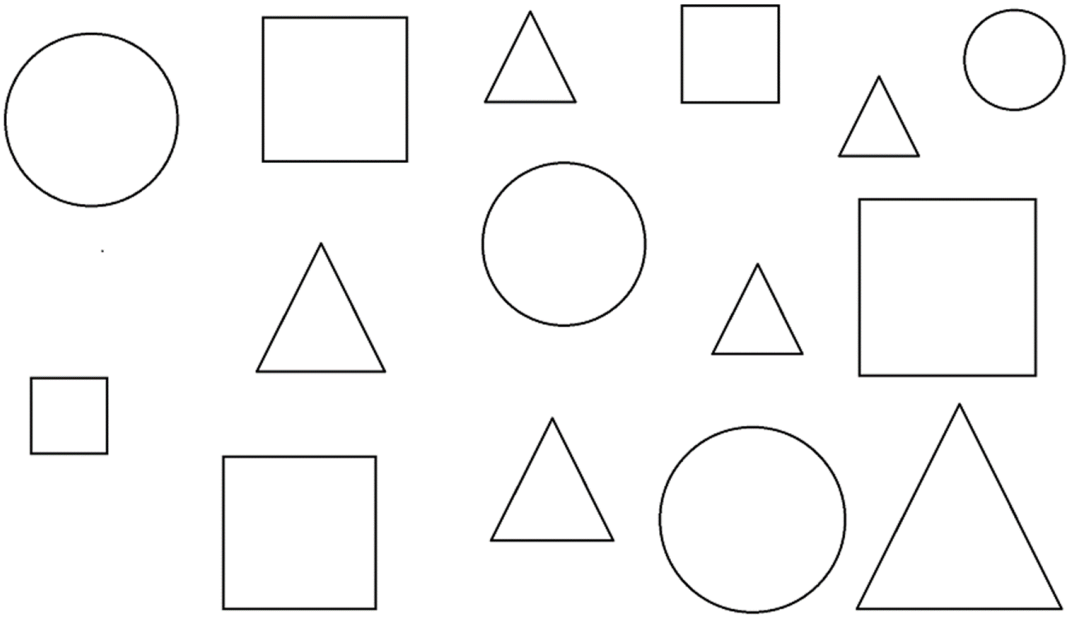

Рисунок 11 – Изображение с фигурами

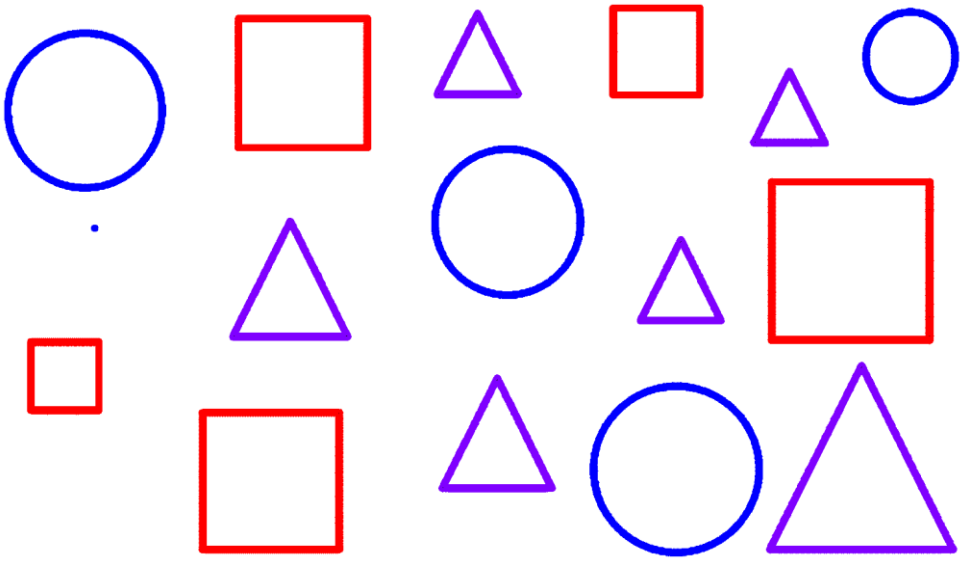

Рисунок 12 – Изображение с фигурами, выделенными цветом

Вывод

В ходе выполнения лабораторной работы были изучены и освоены способы обнаружения линий и окружностей на изображениях с помощью преобразований Хафа, а также методы определения контуров и действий с ними (в данном случае – распознавание формы)